BÀI TẬP TÌM KIẾM TRONG AI

Họ và tên	Hồ Hoàng Khang
MSSV / Lớp	3123410150

Mục tiêu học tập

• Mô phỏng các thuật toán BFS, DFS và Iterative Deepening.

• Biểu diễn bài toán Tháp Hà Nội dưới dạng bài toán tìm kiếm.

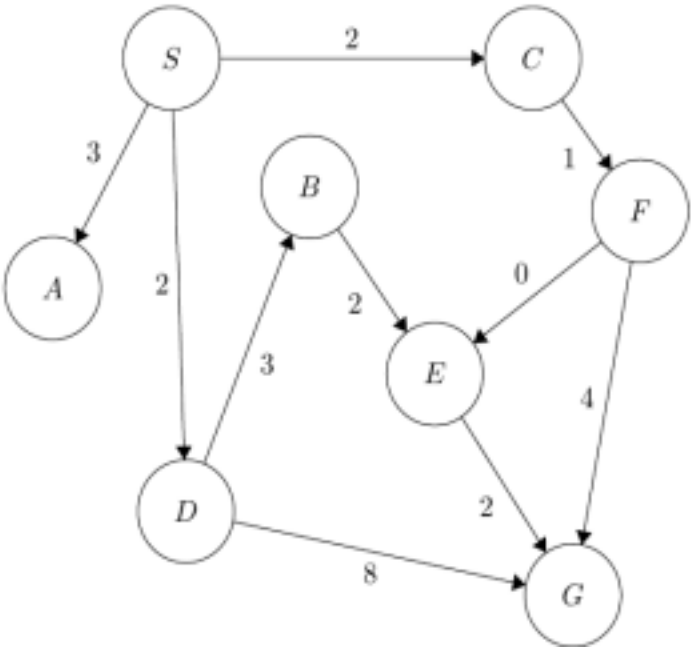
• Thiết kế và phân tích heuristic cho A*.

• So sánh DFS, BFS, UCS, Greedy Search và A* trên cùng một bài toán.

• Phân tích các phát biểu lý thuyết về heuristic, A*, Minimax và Expectimax.

Bài 1. Các thuật toán tìm kiếm

Cho đồ thị có trạng thái bắt đầu S và trạng thái đích G như hình dưới đây.



Hình 1. Đồ thị dùng cho các thuật toán tìm kiếm

Yêu cầu: Với mỗi thuật toán, hãy ghi thứ tự các nút được đưa vào explored set, đường đi được trả về và tổng chi phí của đường đi. Nếu có hòa, phá hòa theo thứ tự alphabet của trạng thái cuối cùng trong đường đi.

1.1. Breadth-First Search (BFS)

Nội dung	Kết quả
Explored Set	S, A, C, D, F, B, G
Path	$S \rightarrow D \rightarrow G$
Total Cost	10

1.2. Depth-First Search (DFS)

Nội dung	Kết quả
Explored Set	S, A, C, F, E, G
Path	$S \rightarrow C \rightarrow F \rightarrow E \rightarrow G$
Total Cost	5

1.3. Iterative Deepening Search (IDS)

Giới hạn độ sâu	Các nút được duyệt
0	S
1	$S \rightarrow A,\ S \rightarrow C,\ S \rightarrow D$
2	$S \rightarrow A,\ S \rightarrow C \rightarrow F,\ S \rightarrow D \rightarrow B,\ S \rightarrow D \rightarrow G$
...	

Path cuối cùng: $S \rightarrow D \rightarrow G$

Total cost: 2 + 8 = 10

thảo luận: BFS và DFS khác nhau ở cấu trúc frontier và tính tối ưu như thế nào?

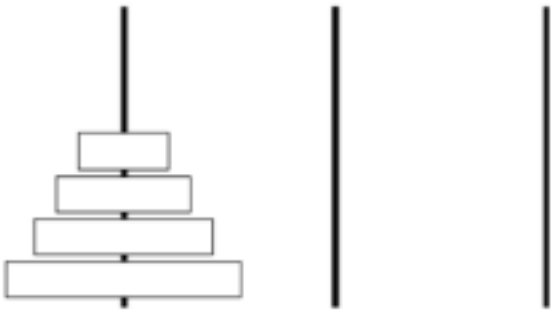
1. **Cấu trúc Frontier (Tập biên / Hàng đợi trạng thái):**
   - **BFS:** Sử dụng cấu trúc **FIFO** (First-In, First-Out - Vào trước ra trước), hoạt động như một hàng đợi (Queue). Các nút được thêm vào cuối hàng đợi và lấy ra từ đầu hàng đợi, giúp thuật toán mở rộng lan rộng theo từng tầng (theo chiều rộng).
   - **DFS:** Sử dụng cấu trúc **LIFO** (Last-In, First-Out - Vào sau ra trước), hoạt động như một ngăn xếp (Stack). Các nút mới sinh ra được đưa lên đầu frontier, khiến thuật toán luôn ưu tiên đi sâu nhất có thể vào một nhánh trước khi quay lui.
2. **Tính tối ưu (Optimality):**
   - **BFS: Tối ưu** (đảm bảo tìm được đường đi ngắn nhất về số bước hoặc chi phí khi các cạnh có

chi phí bằng nhau hoặc không âm) vì nó duyệt qua tất cả các nút ở độ sâu d trước khi chuyển sang độ sâu $d + 1$.

- **DFS: Không tối ưu**. DFS không quan tâm đến độ dài đường đi hay chi phí, nó chỉ cần tìm thấy đích nhanh nhất có thể. Do đó, DFS có thể chọn phải một đường đi rất dài hoặc tốn kém nếu nhánh đó nằm sâu về phía bên trái của cây tìm kiếm.

Bài 2. Tháp Hà Nội - Tower of Hanoi

Bài toán gồm n đĩa có kích thước khác nhau và ba cọc. Ban đầu các đĩa được xếp trên một cọc theo thứ tự từ lớn đến nhỏ. Mỗi lần chỉ được di chuyển một đĩa và không được đặt đĩa lớn lên trên đĩa nhỏ. Mục tiêu là chuyển toàn bộ đĩa sang cọc thứ ba.



Hình 2. Trạng thái ban đầu của bài toán Tháp Hà Nội

2.1. Biểu diễn trạng thái

Hãy đề xuất cách biểu diễn một trạng thái của bài toán. Biểu diễn phải cho biết mỗi đĩa đang ở cọc nào và đủ để xác định các hành động hợp lệ.

- Biểu diễn bằng một bộ (tuple) gồm n phần tử $(p_1, p_2, \cdots, p_n)$, trong đó $p_i \in 1, 2, 3$ là số hiệu cọc mà đĩa i (từ nhỏ đến lớn, 1 là nhỏ nhất, n là lớn nhất) đang đứng.

2.2. Kích thước không gian trạng thái

Với n đĩa, hãy xác định số trạng thái có thể có theo cách biểu diễn của bạn và giải thích cách suy luận.

- Mỗi đĩa có 3 sự lựa chọn về cọc, do đó tổng số trạng thái lý thuyết là 3^n. Do có ràng buộc kích thước (đĩa nhỏ hơn luôn nằm trên đĩa lớn hơn), số lượng trạng thái hợp lệ thực tế là một tập con của 3^n.

2.3. Trạng thái bắt đầu

Hãy viết trạng thái khởi đầu S0 theo cách biểu diễn đã chọn.

- Tất cả n đĩa đều nằm ở cọc thứ nhất: $(1, 1, \cdots, 1)$.

2.4. Các hành động hợp lệ

Từ một trạng thái bất kỳ, hãy mô tả tập Actions(s). Một hành động hợp lệ phải thỏa mãn các điều kiện nào?

- Di chuyển đĩa trên cùng (đỉnh) của một cọc bất kỳ sang cọc khác, với điều kiện:
    1. Cọc nguồn phải có đĩa.
    2. Cọc đích hoặc là rỗng, hoặc có đĩa ở đỉnh lớn hơn kích thước của đĩa được chuyển (không đặt đĩa lớn lên trên đĩa nhỏ).

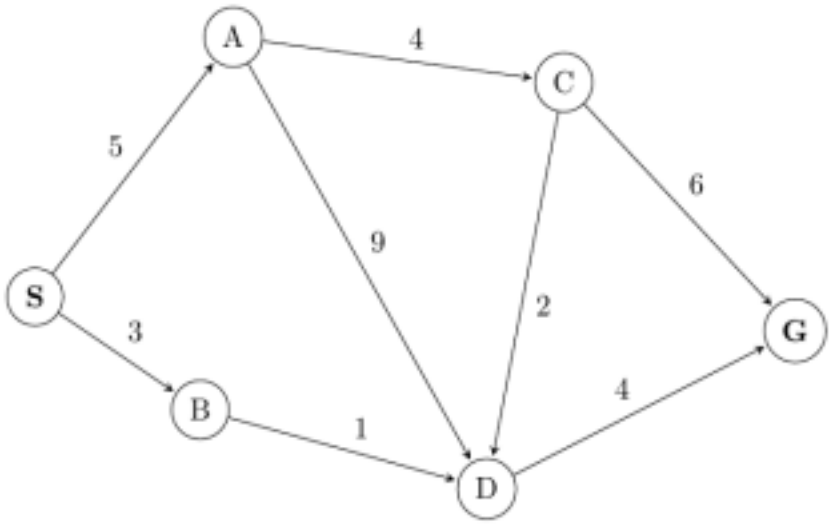
2.5. Goal Test

Hãy mô tả hàm goalTest(state): khi nào hàm trả về True?

- Hàm goalTest(state) trả về True khi toàn bộ n đĩa đã được chuyển sang cọc thứ ba, tức là $(3, 3, \dots, 3)$.

Bài 3. Thiết kế và phân tích Heuristic

Xét đồ thị dưới đây. Hãy thiết kế heuristic và phân tích ảnh hưởng của heuristic đến thuật toán A*.



Hình 3. Đồ thị dùng để thiết kế heuristic

Công thức A*: f(n) = g(n) + h(n), trong đó g(n) là chi phí thực tế từ S đến n và h(n) là chi phí ước lượng từ n đến đích.

3.1. Thiết kế heuristic admissible và consistent

Điền giá trị h(n) cho từng nút sao cho heuristic phù hợp với yêu cầu của nhóm.

Node	h(n)
S	8
A	10
B	5

C	6
D	4
G	0

3.2. Chạy A* với heuristic đã thiết kế

Explored Set: S, B, D, G

Path: S -> B -> D -> G

Total Cost: 8

3.3. Heuristic admissible nhưng không consistent

Đề xuất một bộ giá trị h(n) admissible nhưng không consistent. Chỉ ra ít nhất một cạnh vi phạm điều kiện h(n) <= c(n,n') + h(n').

Đề xuất bộ giá trị h(n): h(S) = 8, h(A) = 10, h(B) = 5, h(C) = 5, h(D) = 4, h(G) = 0.

Bộ giá trị này vẫn admissible vì không vượt quá chi phí thực tế ngắn nhất tới G (h*(A) = 10, h*(C) = 6).

Cạnh vi phạm điều kiện consistency là A -> C:

h(A) = 10 > c(A, C) + h(C) = 4 + 5 = 9.

3.4. Câu hỏi mở rộng

Giải thích vì sao một heuristic consistent thì cũng admissible, giả sử h(G) = 0.

Theo điều kiện consistent (bất đẳng thức tam giác), với mọi cặp nút kề nhau n và n', ta có:

$$h(n) <= c(n, n') + h(n')$$

Áp dụng quy nạp dọc theo đường đi ngắn nhất từ nút n bất kỳ đến nút đích G gồm chuỗi các nút $(n = n_0, n_1, n_2, ..., n_k = G)$:

$$h(n_0) <= c(n_0, n_1) + h(n_1)$$
$$h(n_1) <= c(n_1, n_2) + h(n_2)$$
$$\dots$$
$$h(n_{k-1}) <= c(n_{k-1}, G) + h(G)$$

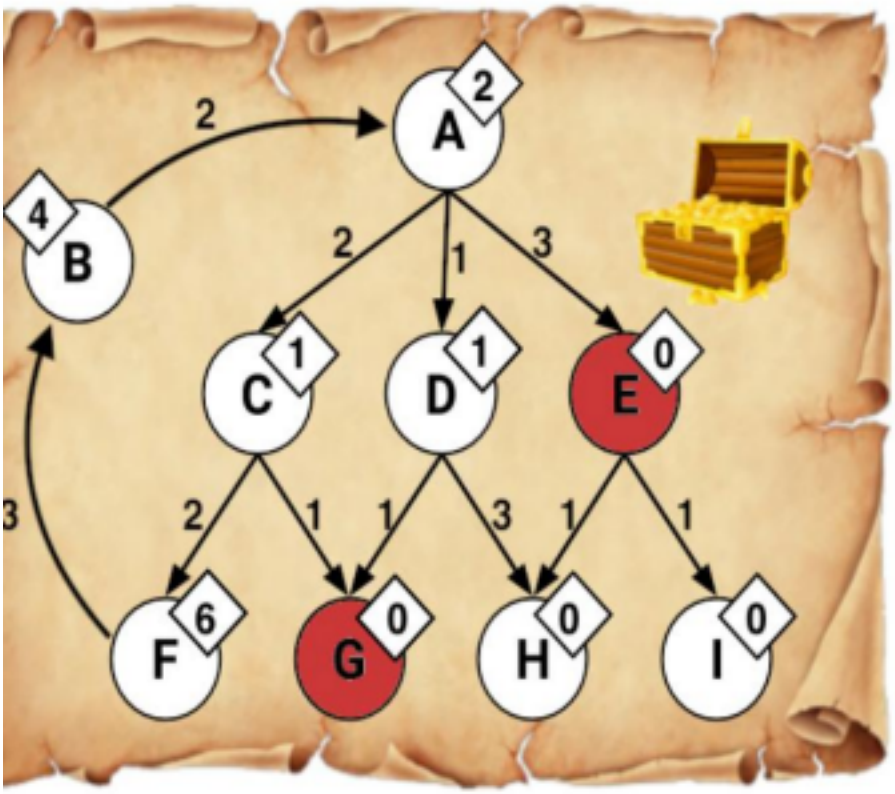
Cộng tất cả các bất đẳng thức trên lại và sử dụng giả thiết h(G) = 0, ta thu được:

$h(n) <= Sum_{i=0}^{k-1} c(n_i, n_{i+1}) = h*(n)$ (trong đó h*(n) là chi phí tối ưu từ n đến G).

Do h(n) <= h*(n) với mọi nút n, heuristic h(n) thỏa mãn tính chất admissible.

Bài 4. Treasure Hunting (Tìm kho báu)

A là đảo xuất phát. Các nút màu đỏ là các đảo có kho báu; chỉ cần tới một trong các đảo đó. Mũi tên biểu diễn hành động có hướng; số trên cạnh là action cost. Giá trị trong hình thoi là heuristic ước lượng chi phí tối thiểu từ nút hiện tại tới một kho báu.



Hình 4. Bản đồ tìm kho báu (giữ nguyên hình gốc)

Quy tắc phá hòa: khi lấy phần tử khỏi frontier, nếu hòa thì chọn theo thứ tự alphabet.

4.1. Depth-First Search

Explored Set: A, C, F, B, D, G, H, E

Path returned: A -> E

Cost: 3

4.2. Breadth-First Search

Explored Set: A, C, D, E

Path returned: A -> E

Cost: 3

4.3. Uniform-Cost Search

Explored Set: A, D, C, G

Path returned: A -> D -> G

Cost: 2

4.4. Greedy Search

Explored Set: A, E

Path returned: A -> E

Cost: 3

4.5. A* Search

Explored Set: A, D, G

Path returned: A -> D -> G

Cost: 2

Bảng tổng hợp so sánh

Thuật toán	Tiêu chí chọn node	Path	Cost
DFS	Độ sâu	A -> E	3
BFS	Độ sâu nhỏ nhất	A -> E	3
UCS	g(n)	A -> D -> G	2
Greedy	h(n)	A -> E	3
A*	g(n)+h(n)	A -> D -> G	2

Bài 5. Câu hỏi Đúng/Sai

Với mỗi phát biểu, hãy ghi Đúng hoặc Sai và giải thích ngắn gọn. Nếu sai, nên đưa ra phản ví dụ.

Câu 1. Depth-first search luôn mở rộng ít nhất nhiều node bằng A* sử dụng một admissible heuristic. Đúng / Sai: Sai

Giải thích: DFS có thể tìm thấy một đích rất nhanh nếu đích nằm ở nhánh bên trái cùng của cây tìm kiếm, dẫn đến số lượng nút mở rộng rất ít. Trong khi đó, A* với heuristic admissible phải đảm bảo tính tối ưu nên đôi khi phải duyệt qua nhiều nhánh có chi phí nhỏ khác trước khi tìm đến đích, do đó DFS không nhất thiết lúc nào cũng mở rộng nhiều nút hơn A*.

Câu 2. Giả sử một quân xe trong cờ vua có thể đi bất kỳ số ô theo hàng hoặc cột trong một nước đi và không thể nhảy qua quân khác. Manhattan distance là một admissible heuristic cho số nước đi nhỏ nhất từ ô A đến ô B. Đúng / Sai: Đúng.

Giải thích: Khoảng cách Manhattan tính tổng số bước di chuyển theo chiều dọc và ngang tối thiểu giữa hai ô trên bàn cờ. Vì quân xe có thể đi dọc hoặc ngang bất kỳ số ô nào trong một nước đi (nếu không bị cản trở), khoảng cách Manhattan chính là số nước đi lý thuyết tối thiểu (hoặc nhỏ hơn thực tế nếu bị vướng quân cản), thỏa mãn tính admissible ($h(n) \leq h^*(n)$).

Câu 3. Euclidean distance là một admissible heuristic cho bài toán tìm đường của

Pacman. Đúng / Sai: Đúng

Giải thích: Khoảng cách Euclid (đường chim bay) là khoảng cách ngắn nhất giữa hai điểm trên mặt phẳng. Vì đường đi thực tế của Pacman luôn lớn hơn hoặc bằng khoảng cách đường chim bay (do bị vướng tường, mê cung), nên $h(n)$ luôn nhỏ hơn hoặc bằng chi phí thực tế tới đích, thỏa mãn tính admissible.

Câu 4. Tổng của nhiều admissible heuristics vẫn luôn là một admissible heuristic.

Đúng / Sai: Sai.

Giải thích:

Câu 5. Admissibility của một heuristic cho A* kéo theo consistency.

Đúng / Sai:

Giải thích: Nếu ta cộng nhiều heuristic admissible lại với nhau (ví dụ: $h(n) = h_1(n) + h_2(n)$), tổng giá trị này có thể vượt quá chi phí thực tế tối ưu $h^*(n)$ đến đích (vi phạm tính admissible). Tuy nhiên, hàm *lấy max* của các admissible heuristics ($h(n) = \max(h_1(n), h_2(n))$) thì luôn admissible.

Câu 6. A* với graph search luôn tối ưu nếu heuristic admissible.

Đúng / Sai: Sai

Giải thích: Để A* với *Graph Search* (tìm kiếm trên đồ thị có lưu tập explored set) đảm bảo tính tối ưu, heuristic bắt buộc phải **consistent** (nhất quán). Nếu heuristic chỉ **admissible** mà không consistent, A* graph search có thể đóng (close) một nút sớm với chi phí chưa tối ưu và không cập nhật lại đường đi khi tìm thấy đường tốt hơn, dẫn đến mất tính tối ưu (tính tối ưu chỉ luôn đúng khi dùng *Tree Search* với heuristic admissible).

Tổng kết nhanh

Nhóm	Thuật toán	Hàm/tiêu chí chính
Uninformed Search	BFS / DFS / IDS	Không sử dụng heuristic
Cost-based Search	UCS	g(n)
Informed Search	Greedy	h(n)
Informed Search	A*	g(n)+h(n)

Ghi nhớ: A* dùng f(n) = g(n) + h(n).

---o0o---

(Hết)